```
#define TILE_SIZE 16


__kernel void bmm_permute_tiled(
    __global const float* A,
    __global const float* B,
    __global float* C,
    const int M,
    const int K,
    const int N,
    const int batch_stride_A,
    const int batch_stride_B,
    const int batch_stride_C,
    const int transA, // 1 если A нужно транспонировать [Batch, K, M] -> [Batch, M, K]
    const int transB  // 1 если B нужно транспонировать [Batch, N, K] -> [Batch, K, N]
) {
    int batch_idx  = get_global_id(2);
    int global_col = get_global_id(0);
    int global_row = get_global_id(1);
    int local_col  = get_local_id(0);
    int local_row  = get_local_id(1);

    // +1 устраняет Bank Conflicts в локальной памяти при чтении столбцов
    __local float Asub[TILE_SIZE][TILE_SIZE + 1];
    __local float Bsub[TILE_SIZE][TILE_SIZE + 1];

    __global const float* current_A = A + (batch_idx * batch_stride_A);
    __global const float* current_B = B + (batch_idx * batch_stride_B);
    __global float* current_C       = C + (batch_idx * batch_stride_C);

    float acc = 0.0f;
    int num_tiles = (K + TILE_SIZE - 1) / TILE_SIZE;

    // Предвычисляем условия, которые не меняются внутри цикла по t
    int is_trans_A  = (transA != 0);
    int is_trans_B  = (transB != 0);
    int valid_row_A = (global_row < M);
    int valid_col_B = (global_col < N);

    for (int t = 0; t < num_tiles; t++) {
        // 1. Бесветвящаяся загрузка тайла из A
        int a_col   = t * TILE_SIZE + local_col;
        int valid_a = valid_row_A && (a_col < K);
        int idx_a   = select(global_row * K + a_col, a_col * M + global_row, is_trans_A);
        int safe_a  = select(0, idx_a, valid_a);
        Asub[local_row][local_col] = select(0.0f, current_A[safe_a], valid_a);

        // 2. Бесветвящаяся загрузка тайла из B
        int b_row   = t * TILE_SIZE + local_row;
        int valid_b = (b_row < K) && valid_col_B;
        int idx_b   = select(b_row * N + global_col, global_col * K + b_row, is_trans_B);
```

```
        int safe_b  = select(0, idx_b, valid_b);
        Bsub[local_row][local_col] = select(0.0f, current_B[safe_b], valid_b);

        barrier(CLK_LOCAL_MEM_FENCE);

        #pragma unroll
        for (int k = 0; k < TILE_SIZE; k++) {
            acc += Asub[local_row][k] * Bsub[k][local_col];
        }

        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if (valid_row_A && valid_col_B) {
        current_C[global_row * N + global_col] = acc;
    }
}
```

Я несовсем понимаю, есть ли в этой задаче зависимости вида xi=xi_-1 . Если их нет, в теори отказавшись о т лдокальной памяти можно вообще выкинуть барьеры и считает двумя паралельными осями? (распараллелить цикл numtiles)?

```
public Tensor BmmPermute(Tensor a, Tensor b, bool transA = false, bool transB = false)
{
    if (a.Shape.Length != 3 || b.Shape.Length != 3)
        throw new ArgumentException("BmmPermute ожидает 3D-тензоры.");

    int batch = a.Shape[0];
    int m = transA ? a.Shape[2] : a.Shape[1];
    int kA = transA ? a.Shape[1] : a.Shape[2];
    int kB = transB ? b.Shape[2] : b.Shape[1];
    int n = transB ? b.Shape[1] : b.Shape[2];

    if (kA != kB)
        throw new ArgumentException($"Несовпадение внутренних осей: {kA} != {kB}.");

    int[] outShape = { batch, m, n };
    Tensor c = new Tensor(outShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);

    int batchStrideA = (a.Shape[0] == 1) ? 0 : (a.Shape[1] * a.Shape[2]);
    int batchStrideB = (b.Shape[0] == 1) ? 0 : (b.Shape[1] * b.Shape[2]);
    int batchStrideC = m * n;

    int gx = (int)Math.Ceiling(n / 16.0) * 16;
    int gy = (int)Math.Ceiling(m / 16.0) * 16;

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

    int t = 0;
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(a.DataBuffer);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(b.DataBuffer);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(c.DataBuffer);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(m);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(kA);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(n);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(batchStrideA);
```

```
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(batchStrideB);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(batchStrideC);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(transA ? 1 : 0);
    _kernelBmmPermuteTiled.Arguments[t++].SetValue(transB ? 1 : 0);

    activeQueue.EnqueueNDRangeKernel(
        _kernelBmmPermuteTiled,

        new[] { new IntPtr(gx), new IntPtr(gy), new IntPtr(batch) },
        new[] { new IntPtr(16), new IntPtr(16), new IntPtr(1) }
    );
    //activeQueue.Finish();

    return c;
}
```